Reliability, Consistency, and Threading Architectures in Cloud Message Brokers: A Distributed-Systems Performance Study

Bhuvana Komati[1], Himanshu Jain[2], and Vidhyacharan Bhaskar[3]*

[123]Department of Computer Engineering

San Jose State University

One Washington Square, San Jose, CA 95192, USA

Email: bhuvana.komati@sjsu.edu, himanshu.jain@sjsu.edu, charan.bhaskar@sjsu.edu

**Corresponding author email: charan.bhaskar@sjsu.edu*

October 2026

Abstract

Cloud message brokers such as AWS SQS, RabbitMQ, and Apache Kafka must guarantee at-least-once delivery, durability, ordering, and—for transactional workloads—atomicity and agreement across replicas. These mechanisms, and the thread-pool architecture that executes them, impose a performance cost that classical queueing models ignore. This companion to our study of heavy-tailed service times holds the workload fixed and quantifies the cost of the broker's distributed-systems machinery with a seeded SimPy discrete-event framework, reporting every result as a mean over independent replications with a 95% confidence interval and, wherever a closed form exists, against an exact analytical value. Two-Phase Commit with three replicas, 10 ms round trips, and 0.99 replica availability adds 60.5 ms per message—half of it from rare 1 s vote timeouts—and cuts saturation throughput by 42.3%; Raft commit on the critical path costs 7.0% of capacity on a LAN but 36.2% on a WAN. The visibility timeout has an interior optimum: very short timeouts cause duplicate deliveries (54% of messages at 0.5 s) and dead-lettering, while long ones add roughly a tenth of the timeout to mean latency. Replication matters only when nodes fail: with node outages, two replicas cut the mean time to first delivery from 4.92 s to 0.50 s. Strict in-order delivery costs 19% mean latency at moderate load. A shared thread pool is 10% slower than an ideal pool at $\rho = 0.7$ and 4.8× slower at $\rho = 0.9$, whereas a dedicated pool behaves as an Erlang loss system that rejects up to 49% of arrivals. For heterogeneous servers we show that the large penalties often attributed to service-rate variance are instead per-pool instability caused by load-oblivious dispatch: with capacity-proportional or queue-aware routing every configuration is stable, and the most heterogeneous one waits no longer than a homogeneous fleet.

Keywords: Two-Phase Commit, Raft Consensus, Vector Clocks, Causal Delivery, Thread Pool Architecture, Visibility Timeout, Replication, Message Ordering, Heterogeneous Servers, Discrete-Event Simulation

1 Introduction

Cloud message brokers decouple producers from consumers and absorb load spikes through asynchronous queues. Production systems such as AWS Simple Queue Service (SQS), RabbitMQ, and Apache Kafka are not simple FIFO buffers: to guarantee no message loss, survivable storage, ordered or causally consistent delivery, and in some cases transactional atomicity, they layer visibility timeouts and redelivery, multi-replica storage, consensus, and atomic-commit protocols on top of the raw queue. Each mechanism consumes time and threads, and that cost is invisible to a textbook M/M/N analysis.

In a companion paper we studied one axis of broker performance—the effect of heavy-tailed (Pareto) service-time distributions on tail latency and the resulting gap between M/M/N predictions and simulation. The present paper is the orthogonal study: we hold the service distribution (exponential, unless stated) fixed and quantify the performance consequences of the reliability, consistency, and concurrency mechanisms themselves.

Our contributions are fourfold. First, we measure the trade-offs of three reliability mechanisms—visibility timeout, replication under node outages, and message ordering. Second, we measure the latency and capacity cost of Two-Phase Commit and Raft on the critical path, and the hold-back cost of vector-clock causal delivery. Third, we compare dedicated and shared thread pools and identify each as a classical queueing system. Fourth, we relax four baseline assumptions—homogeneous servers, infinite buffers, a single priority class, and exponential service—and validate each extension against an exact or standard analytical result.

1.1 Background and Related Work

Lamport's logical clocks [2] established the happened-before relation, later refined into vector clocks by Fidge [3] and Mattern [4]; causal broadcast built on them was given by Birman, Schiper, and Stephenson [22]. Atomic commitment was formalized by Gray [7] as Two-Phase Commit, whose blocking behavior was analyzed by Gray and Lamport [8]. Fischer, Lynch, and Paterson [11] showed consensus impossible in a purely asynchronous system with one crash, motivating leader-based protocols such as Paxos [6] and Raft [5]. Brewer's CAP conjecture [9], proved by Gilbert and Lynch [10], frames the choice between consistency and availability under partition; Vogels [12] and Dynamo [13] describe the eventually consistent alternative. Dean and Barroso [14] showed that tail latency governs user experience at scale, which motivates our P99 measurements.

On the queueing side we use Erlang's loss and delay formulas [16], Kleinrock's treatment [15], Little's Law [17], Cobham's priority formula [23], the Allen–Cunneen approximation [24], and Harchol-Balter's synthesis [18]. Our base system is the cloud-broker model of Li et al. [1], whose visibility-timeout and ordering semantics we implement; where Li et al. use Colored Petri Nets, we use discrete-event simulation in SimPy [20].

2 System Model and Methodology

All results are produced by a discrete-event simulator written in Python with SimPy 4.1.1 [20]. Messages arrive as a Poisson process of rate λ and are served by threads modeled as SimPy resources; protocols are modeled either as explicit state machines (storage nodes with visibility-timeout managers, Raft nodes, causal-broadcast members) or as additional service-time components (2PC). Every run discards a warm-up interval, and every random stream is seeded. Unless stated otherwise each configuration is replicated 10 times with independent seeds, and we report the mean across replications with a 95% Student-t confidence interval (written mean ± half-width). Table 1 lists the common parameters.

Table 1: Common system and protocol parameters.

Parameter	Symbol	Default value
Service rate per thread	μ	12 msg/s
2PC round-trip time	RTT	10 ms (σ = 2 ms)
2PC replica availability / vote timeout	a, T	0.99, 1.0 s
2PC replicas	n	3
Raft LAN: election timeout, heartbeat, one-way delay	—	U[150, 300] ms, 50 ms, 2 ms + U[0, 3] ms

Raft WAN: election timeout, heartbeat, one-way delay	—	U[150, 300] ms, 50 ms, 20 ms + U[0, 10] ms
Visibility timeout (SQS default)	VT	30 s
Max receive count (dead-letter threshold)	—	3
Shared-pool overhead coefficient	α	0.1
Threads per dedicated connection	—	2

Model verification. Wherever an exact result exists we compare the simulator against it: M/M/N (Erlang-C) for the threading baseline, Erlang-B for the dedicated pool, independent M/M/n_i queues for heterogeneous servers under static dispatch, the M/M/N/K birth–death solution for finite buffers, Cobham's formula and the work-conservation law for priority classes, a numerically solved continuous-time Markov chain (CTMC) for Erlang-k service, the closed-form 2PC overhead moments, and $(1 - f)^n$ for the 2PC commit rate. These checks are reported alongside each result.

3 Reliability Mechanisms

The broker model stores each message on a set of storage nodes chosen by a consistent-hash ring (150 virtual nodes per physical node, MD5 placement). Consumers poll a randomly chosen node—modeling SQS short polling—and back off for 0.1 s if it has no visible message. Receiving a copy hides all replicas of that message, so each logical message is delivered to one consumer at a time; acknowledgements use receipt handles, so an acknowledgement issued for a delivery that has already timed out is rejected, as in SQS.

3.1 Visibility Timeout and Redelivery

A received message is hidden for VT seconds. If it is acknowledged in time it is deleted; otherwise it becomes visible again and is redelivered, and a message whose receive count reaches 3 is moved to a dead-letter queue (DLQ) [19]. VT therefore has opposing effects: if it is shorter than the processing time, healthy consumers are timed out and the message is processed twice; if it is long, a genuinely failed message waits VT before its retry. We swept VT from 0.5 s to 120 s with $\lambda = 10$ msg/s, exponential processing with mean 0.5 s, failure probability 0.1, 10 consumers, three nodes, and replication factor two (2000 s runs, 200 s warm-up, 10 replications). Duplicate rate is the number of completed-but-stale deliveries per published message; the delivery and DLQ ratios are measured over messages published at least $3 \cdot \mathrm{VT} + 10$ s before the end of the run.

Table 2: Effect of visibility timeout (mean ± 95% CI over 10 replications).

VT (s)	Mean latency (s)	P99 latency (s)	Mean retries	Duplicates	Dead-lettered
0.5	0.77 ± 0.03	3.6 ± 0.3	0.500	53.66%	7.94%
1	0.67 ± 0.01	2.8 ± 0.0	0.252	15.48%	1.13%
2	0.77 ± 0.01	4.2 ± 0.0	0.128	1.85%	0.16%
5	1.09 ± 0.01	7.6 ± 0.7	0.109	0.01%	0.10%
10	1.63 ± 0.02	13.3 ± 1.8	0.108	0.00%	0.11%
30	3.80 ± 0.05	36.8 ± 7.1	0.108	0.00%	0.10%
60	7.07 ± 0.12	68.1 ± 13.1	0.109	0.00%	0.10%
120	13.61 ± 0.20	122.5 ± 0.3	0.109	0.00%	0.10%

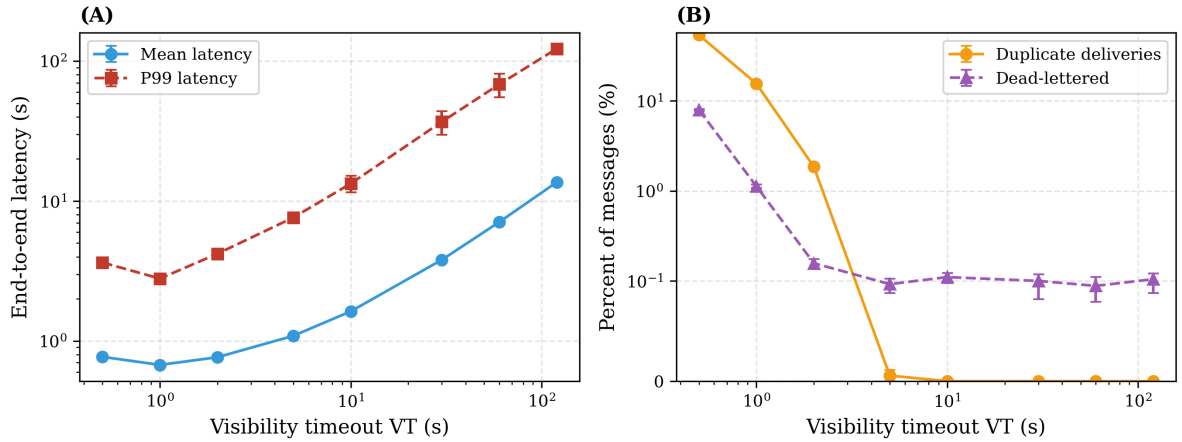


Figure 1: Visibility-timeout trade-off. (A) Mean and P99 latency are minimized near VT = 1 s. (B) Below about 5 s, premature timeouts cause duplicate processing and dead-lettering.

Mean latency is minimized at VT = 1 s (0.675 s). Shorter timeouts expire while healthy consumers are still working: at VT = 0.5 s, 53.7% of messages are processed twice and 7.9% are wrongly dead-lettered after three premature timeouts. Once VT exceeds the processing time by a wide margin (VT ≥ 5 s), duplicates disappear (0.01%) and the DLQ ratio settles at 0.10%, consistent with the $0.1^3 = 0.1\%$ probability of three consecutive genuine failures. Beyond that point latency grows linearly with VT, because about one message in ten waits a full timeout before its retry: mean latency rises from 1.09 s at 5 s to 3.80 s at the SQS default of 30 s and 13.61 s at 120 s, and P99 latency tracks VT itself (36.8 s and 122.5 s). Mean retries fall only until duplicates vanish and then stay near the failure probability. VT should therefore be set a small multiple above the processing-time tail; the 30 s default is safe but costs latency for sub-second workloads.

3.2 Replication Under Node Outages

Each message is written to its primary node and the next R − 1 nodes on the ring, so storage grows R-fold. Without node failures replication can only add cost, so we also inject independent node outages: each of the three nodes fails after an exponential up-time with mean 100 s and recovers after an exponential down-time with mean 20 s. While a node is down its copies cannot be read; acknowledgements are still applied. We used $\lambda = 100$ msg/s, processing mean 0.05 s, failure probability 0.1, VT = 30 s, and 20 consumers (enough that polls wasted on a down node do not saturate the consumer pool), with 500 s runs, a 50 s warm-up, and 30 replications. Because end-to-end latency is dominated by the 30 s retry of the 10% failed messages, we also report the time from publication to first delivery, which isolates availability.

Table 3: Replication factor with and without node outages (30 replications).

Outages	R	Copies / msg	Mean latency (s)	Mean time to 1st delivery (s)	P99 time to 1st delivery (s)	Throughput (msg/s)
off	1	1.0	3.31 ± 0.02	0.020 ± 0.000	0.11 ± 0.00	99.9
off	2	2.0	3.30 ± 0.02	0.012 ± 0.000	0.06 ± 0.00	99.8
off	3	3.0	3.29 ± 0.02	0.010 ± 0.000	0.05 ± 0.00	99.9
on	1	1.0	8.58 ± 1.08	4.922 ± 0.997	62.40 ± 7.79	99.5
on	2	2.0	3.83 ± 0.31	0.502 ± 0.281	13.25 ± 6.59	99.9
on	3	3.0	3.93 ± 0.75	0.597 ± 0.704	8.28 ± 6.86	99.8

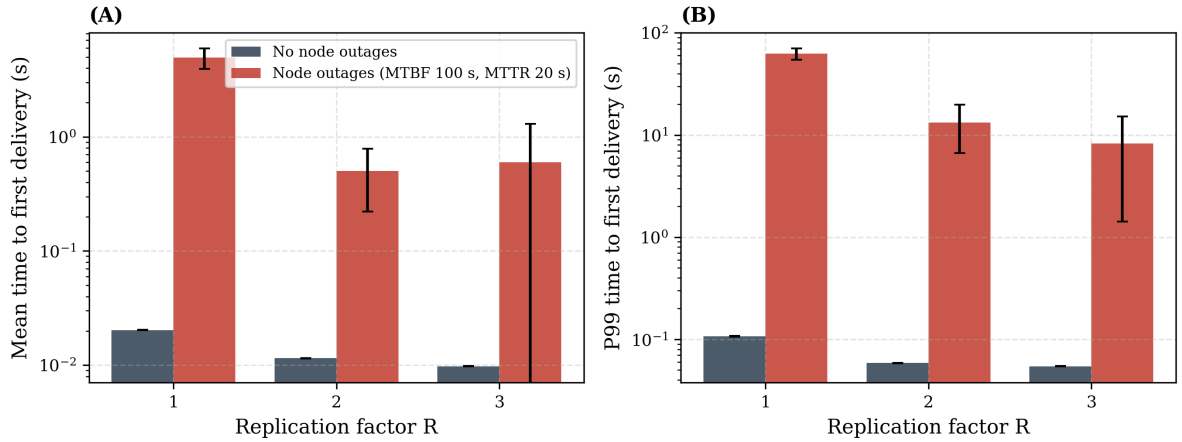


Figure 2: Time from publication to first delivery versus replication factor, with and without node outages (log scale, 95% CIs).

Without outages, the replication factor has no material effect on latency or throughput (mean latency 3.29–3.31 s) while storage grows 3-fold. With outages a single copy is unreadable whenever its node is down, so the mean time to first delivery rises to 4.92 ± 1.00 s and its P99 to 62.4 s. A second replica reduces these to 0.50 ± 0.28 s and 13.3 s, and mean end-to-end latency from 8.58 s to 3.83 s. A third replica gives no further improvement that is distinguishable at 30 replications (0.60 ± 0.70 s), because with three nodes the probability that two are down at once is small. Throughput stays at the offered load in every case. In this model the benefit of replication is availability during outages, and the second copy captures nearly all of it.

3.3 Message Ordering

In out-of-order mode a consumer may take any visible message; in in-order mode it may take only the message whose producer sequence number is next, so a delayed message blocks those behind it (head-of-line blocking), following Li et al. [1]. We used one storage node, five consumers that poll every 1 ms, Pareto($\alpha = 1.5$) processing times with minimum 10 ms (mean 30 ms, consumer capacity $\approx$ 167 msg/s), and a uniform 10–100 ms network delay between producer and broker that reorders messages. Latency is measured from production to completion and includes the network delay; λ was swept from 40 to 150 msg/s (500 s runs, 50 s warm-up, 10 replications).

Table 4: In-order versus out-of-order delivery (mean ± 95% CI, 10 replications).

λ (msg/s)	Out-of-order mean (ms)	In-order mean (ms)	Out-of-order P99 (ms)	In-order P99 (ms)
40	85.1 ± 0.7	94.4 ± 0.4	279 ± 7	283 ± 8
80	87.5 ± 1.3	104.4 ± 1.7	293 ± 10	308 ± 29
120	122.2 ± 17.8	138.3 ± 23.6	891 ± 331	823 ± 665
150	814.0 ± 798.2	443.9 ± 206.8	9146 ± 9516	2495 ± 1089

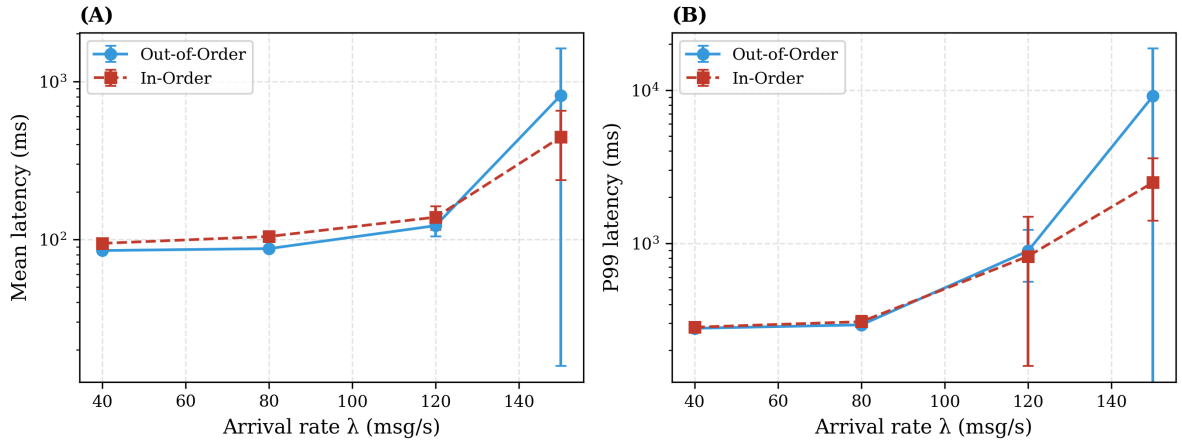


Figure 3: Mean (A) and P99 (B) latency of in-order and out-of-order delivery versus arrival rate (log scale, 95% CIs).

At light and moderate load, in-order delivery adds 9.4 ms (+11%) to the mean at $\lambda = 40$ and 16.9 ms (+19%) at $\lambda = 80$; both differences are far outside their confidence intervals. The penalty is the time a message waits for predecessors still in flight on the reordering network path. The P99 difference at these loads is within the confidence intervals, because the tail is dominated by the infinite-variance Pareto processing time rather than by reordering. At $\lambda = 150$ ($\rho \approx 0.9$) the infinite-variance service makes 10 replications insufficient: the confidence intervals (±798 ms and ±207 ms) overlap completely, and we draw no conclusion at that load.

4 Consistency and Consensus

4.1 Two-Phase Commit

Two-Phase Commit (2PC) [7] makes a transaction atomic across participants. In our protocol model a participant votes YES with probability equal to its availability and the coordinator commits only if every vote is YES. Over 200 transactions and 10 seeds the measured commit rate tracks the analytical $(1 - f)^n$ for participant failure probability f (Table 5), and every participant records the same commit/abort decisions in every replication, confirming atomicity.

Table 5: 2PC commit rate versus participant failure probability (10 replications).

Participants n	Failure prob. f	Measured commit rate	$(1 - f)^n$	Atomicity
3	0	1.000 ± 0.000	1.000	holds
3	0.05	0.855 ± 0.014	0.857	holds
3	0.1	0.733 ± 0.015	0.729	holds
3	0.3	0.369 ± 0.021	0.343	holds
5	0	1.000 ± 0.000	1.000	holds
5	0.05	0.777 ± 0.016	0.774	holds
5	0.1	0.599 ± 0.018	0.590	holds
5	0.3	0.187 ± 0.013	0.168	holds

Its performance cost comes from the critical path. Each message's service time becomes S = S_base + T_2PC, where the prepare and commit phases each cost one round trip and the vote phase waits for the slowest of n replicas, or for the full vote timeout T if any replica is unavailable:

$$T_2PC = RTT_prepare + V + RTT_commit \quad (1)$$

$$E[V] = a^n \cdot E[\max(RTT_1,\ldots,RTT_n)] + (1 - a^n) \cdot T \quad (2)$$

$$\lambda^* = N / E[S] \quad \text{(saturation arrival rate)} \quad (3)$$

With the defaults (n = 3, a = 0.99, T = 1 s) a vote times out with probability $1 - a^3 = 0.0297$, and that rare 1 s timeout contributes about 29.7 ms to the mean—about as much as the three round trips together. Equation (2) gives E[T_2PC] = 61.0 ms, and 2×10^5 sampled overheads give 60.5 ms (Table 6); the overhead variance is likewise dominated by the timeout term (0.0282 s^2 analytical, 0.0277 s^2 sampled, against 0.000010 s^2 with a = 1). Adding E[T_2PC] to the 83.3 ms base service time lowers the saturation rate of a 10-thread pool from 120 to 69.3 msg/s, a 42.3% capacity loss.

Table 6: 2PC overhead per message, sampled (2×10^5 draws) versus analytical, and the resulting saturation rate for N = 10, μ = 12.

n	a	T (s)	Sampled E[T_2PC] (ms)	Analytical (ms)	λ* (msg/s)	Capacity loss
3	0.99	1	60.5	61.0	69.3	42.3%
3	1	1	31.7	31.7	86.9	27.6%
3	0.999	1	34.8	34.7	84.8	29.4%
3	0.95	1	171.7	172.6	39.1	67.4%
3	0.99	0.1	34.3	34.3	85.0	29.2%
3	0.99	0.5	45.9	46.2	77.2	35.7%
1	0.99	1	39.7	39.9	81.1	32.4%
5	0.99	1	81.0	80.7	61.0	49.2%
7	0.99	1	98.6	99.8	54.6	54.5%

We then simulated an M/M/10 queue (μ = 12) whose service time includes the sampled 2PC overhead, at arrival rates for which the effective utilization stays below 0.9 (1000 s runs, 100 s warm-up, 10 replications), and a closed-loop saturated pool to measure capacity directly (Table 7).

Table 7: End-to-end effect of 2PC on an M/M/10 queue (μ = 12) at λ = 60 msg/s, and measured saturation throughput (10 replications).

Configuration	Mean response (s)	P99 response (s)	Saturation throughput (msg/s)
No 2PC	0.0841 ± 0.0003	0.386	119.7 ± 0.5
2PC, a = 1, T = 1 s	0.1211 ± 0.0005	0.425	86.6 ± 0.2
2PC, a = 0.999, T = 1 s	0.1255 ± 0.0007	0.455	84.6 ± 0.3
2PC, a = 0.99, T = 0.1 s	0.1253 ± 0.0005	0.431	84.6 ± 0.3
2PC, a = 0.99, T = 1 s	0.2184 ± 0.0055	1.192	69.0 ± 0.4

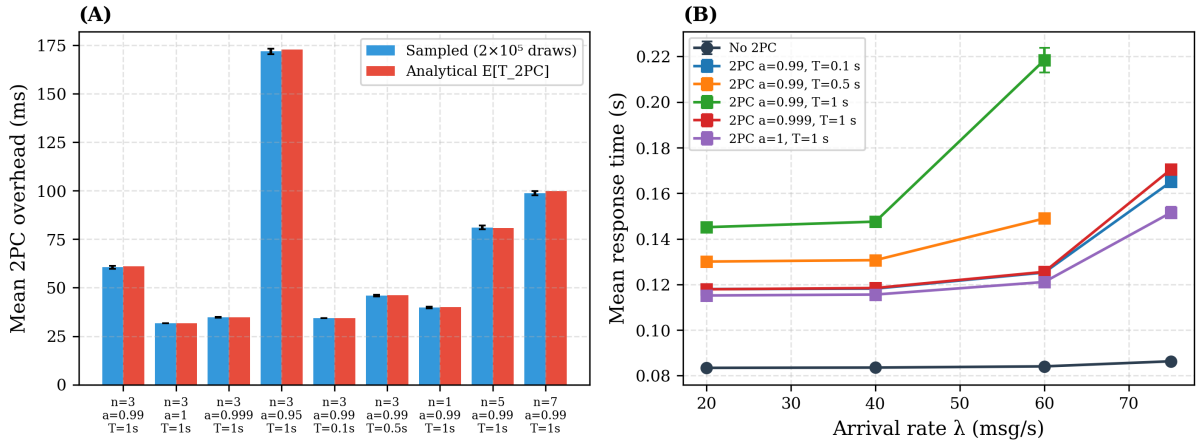


Figure 4: (A) Mean 2PC overhead, sampled versus Eq. (2), across replica counts, availabilities, and vote timeouts. (B) Mean response time of an M/M/10 queue with and without 2PC (95% CIs).

At λ = 60 msg/s the default 2PC configuration raises mean response time from 0.0841 s to 0.2184 s (2.60×) and P99 from 0.386 s to 1.192 s; the P99 sits just above T + 1/μ because the ~3% of transactions that hit a vote timeout form the tail. With perfectly available replicas (a = 1) the mean is 0.1211 s (1.44×). Measured saturation throughput agrees with N/E[S] to within 1% in every configuration. Shortening the vote timeout to 0.1 s recovers most of the lost capacity (84.6 versus 69.0 msg/s), at the price of aborting transactions whose votes are merely slow.

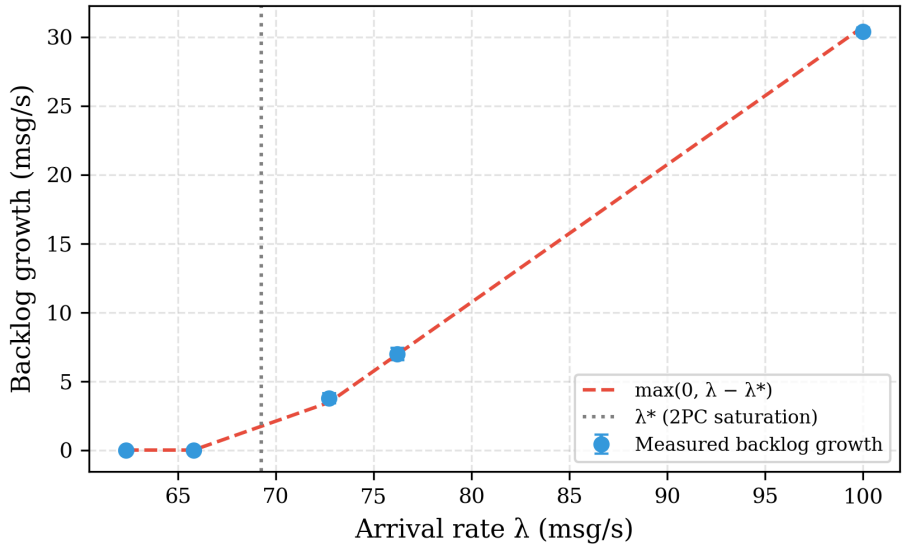


Figure 5: Backlog growth rate of the default 2PC queue versus arrival rate. Above λ* the backlog grows at λ − λ*.

A practical consequence is that a pool that is comfortably stable without 2PC can be unstable with it. Above λ* the measured backlog grows at λ − λ* (Figure 5): at λ = 100 msg/s—ρ = 0.83 without 2PC—the backlog grows at 30.4 ± 0.3 msg/s (predicted 30.7) and throughput saturates at 69.5 msg/s, while the same queue without 2PC serves 100.1 msg/s with no backlog growth.

4.2 Raft Consensus

Raft [5] elects a leader by randomized election timeouts and commits a log entry once it is stored on a majority. Our Raft model exchanges RequestVote and AppendEntries messages over a network with a configurable one-way delay and jitter, and we run it with realistic LAN and WAN parameters (Table 1) for clusters of 3, 5, and 7 nodes. Table 8 reports time to elect the first leader and time to elect a new one after crashing the leader (30 replications each), and the latency from appending an entry at the leader to its majority commit, at 100 appends/s (20 replications).

Table 8: Raft election and commit latency (mean ± 95% CI).

Network	n	First leader (ms)	Re-election after crash (ms)	Commit latency mean (ms)	Commit latency P99 (ms)
LAN	3	203 ± 12	169 ± 22	6.25 ± 0.02	8.5
LAN	5	183 ± 8	133 ± 8	6.54 ± 0.01	8.3
LAN	7	175 ± 5	134 ± 14	6.66 ± 0.01	8.2
WAN	3	270 ± 33	285 ± 48	47.12 ± 0.06	54.3
WAN	5	244 ± 24	251 ± 33	48.03 ± 0.06	53.6
WAN	7	224 ± 13	225 ± 28	48.40 ± 0.04	53.2

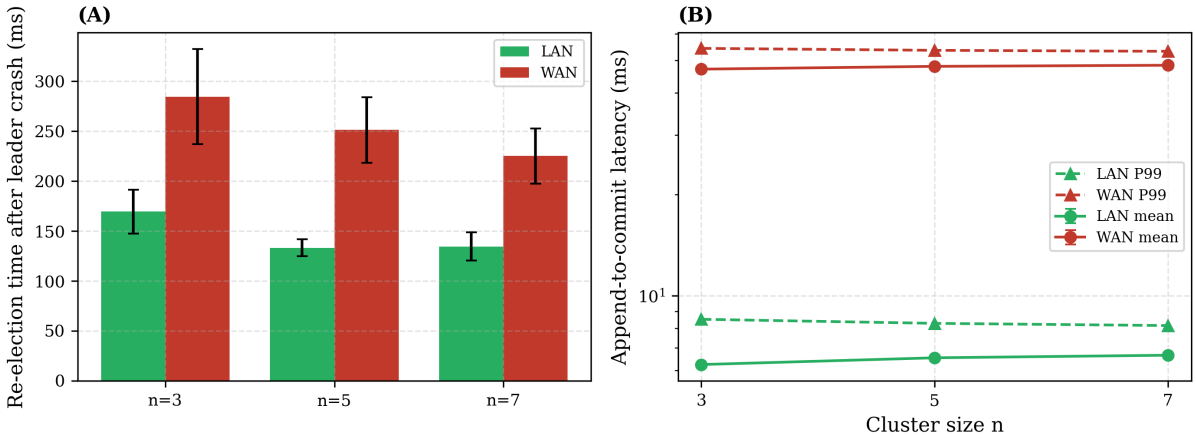


Figure 6: (A) Raft re-election time after a leader crash. (B) Append-to-majority-commit latency (log scale).

Every run elected exactly one leader per term and completed fail-over, with zero election-safety violations. Leader election completes within one to two election-timeout intervals; on the WAN, split votes occurred before the first leader in 4, 3, and 1 of 30 runs for n = 3, 5, 7, lengthening elections. Commit latency is essentially one round trip to the fastest majority: 6.25 ms on the LAN and 47.1 ms on the WAN for n = 3, rising only slightly with cluster size because a larger majority must be waited for.

To measure the cost on the request path, each worker thread of an M/M/10 queue (μ = 12) blocks until its request's log entry commits. At λ = 100 msg/s a 3-node LAN cluster raises mean response time from 0.1076 s to 0.1425 s (1.32×) with unchanged throughput. On the WAN the same cluster costs 1.75× at λ = 60 and makes the queue unstable at λ = 100: measured effective utilization is 1.31 and throughput saturates at 76.6 msg/s. Saturated-pool measurements (Figure 7) give the capacity cost directly: 7.0% (n = 3) and 7.4% (n = 5) on the LAN, 36.2% and 36.7% on the WAN. The cost of consensus is therefore set by network round-trip time relative to service time, not by the protocol itself. This model commits each request synchronously with no batching or pipelining, so these figures are an upper bound for implementations that batch entries.

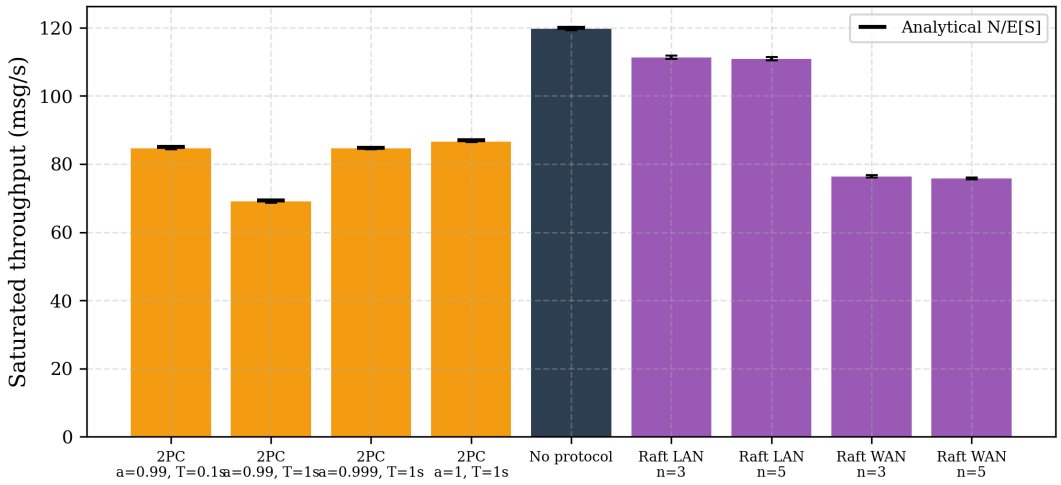


Figure 7: Measured saturation throughput of a 10-thread pool (μ = 12) with no protocol, 2PC, and Raft on the critical path (10 replications); black marks show N/E[S] for 2PC.

4.3 Vector Clocks and Causal Delivery

Vector clocks [3,4] let a broker deliver messages in causal rather than merely FIFO order. We implement the causal broadcast of Birman, Schiper, and Stephenson [22]: process i delivers a message

m from j only when m.VC[j] = VC_i[j] + 1 and m.VC[k] ≤ VC_i[k] for all k ≠ j, holding it back otherwise. P processes each broadcast 10 messages/s for 30 s over links with 10 ms mean lognormal delay whose coefficient of variation (CV) we vary (10 replications). As a control, the same workload is delivered without ordering, and we count deliveries that violate causality against the ground-truth happened-before relation.

Table 9: Causal delivery cost: fraction of messages held back and mean hold-back of held messages, versus delay variability (10 broadcasts/s).

P	Delay CV	Held back	Mean hold-back if held (ms)	P99 hold-back, all (ms)	Violations without ordering
3	0	0.0%	0.0	0.0	0.0%
3	0.5	2.9%	4.5	4.4	2.8%
3	1	6.0%	11.9	19.8	5.6%
3	2	9.5%	32.0	77.9	8.1%
5	0	0.0%	0.0	0.0	0.0%
5	0.5	3.7%	5.0	6.5	3.7%
5	1	9.5%	13.4	31.4	9.1%
5	2	16.2%	40.1	143.4	14.1%
10	0	0.0%	0.0	0.0	0.0%
10	0.5	5.8%	4.7	8.2	5.8%
10	1	16.9%	13.5	41.6	16.9%
10	2	28.4%	41.8	195.2	26.9%

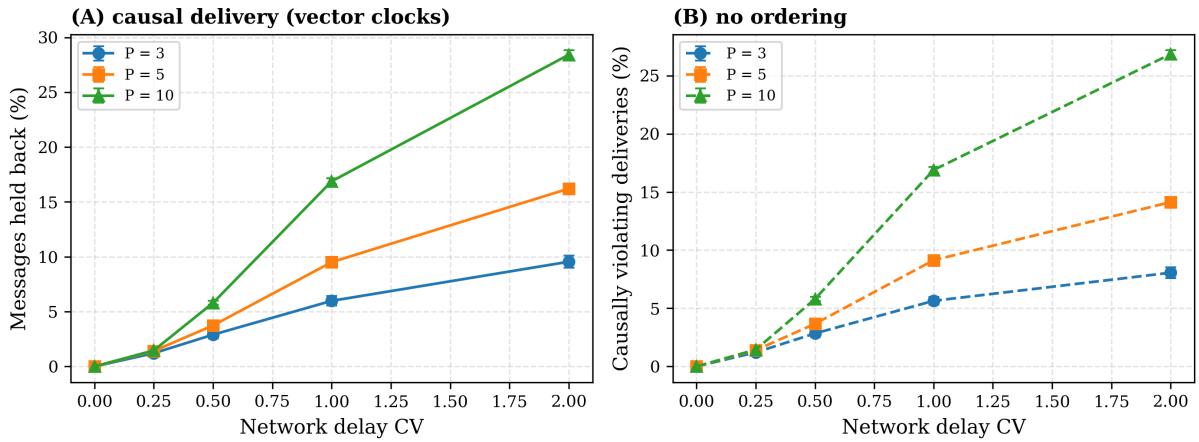


Figure 8: (A) Messages held back by causal delivery and (B) causally violating deliveries without ordering, versus network delay CV.

Causal delivery never violated causality in any run, whereas unordered delivery violated it for up to 26.9% of deliveries. Its cost is hold-back, which is zero when delays are constant and grows with delay variability, group size, and broadcast rate: at CV = 1, 16.9% of messages are held back with P = 10, and at P = 5 the held fraction rises from 1.9% at 2 broadcasts/s to 34.8% at 50/s. The hold-back of an affected message is on the order of the delay spread (about 14 ms at CV = 1), so the mean penalty is small but the tail grows with variability: the P99 hold-back over all messages reaches 195 ms at P = 10 and CV = 2.

4.4 Consistency Choices Under CAP

The CAP theorem [9,10] forces a replicated broker to choose between consistency and availability during a partition. A CP design keeps a single order of updates through consensus or atomic commit, and therefore pays the costs measured above on every operation; an AP design acknowledges locally

and reconciles later, avoiding them at the price of temporarily inconsistent reads. Our measurements put numbers on the CP side of this choice for a 10-thread pool: causal delivery costs only hold-back proportional to delay jitter; Raft costs 7% of capacity on a LAN and 36% on a WAN; and 2PC costs 28% even with perfectly available replicas and 42% with 1% replica unavailability. We do not simulate partitions, so we make no quantitative availability claims.

5 Threading Architectures

We compare two thread-pool architectures with N = 20 threads and μ = 12 msg/s per thread against an ideal M/M/20 pool, sweeping ρ = λ/(Nμ) from 0.5 to 0.9 (2000 s runs, 200 s warm-up, 10 replications). In the dedicated design each connection reserves two threads, so at most N/2 = 10 connections are active and further arrivals are rejected. Each accepted message is served at rate μ, so this design is exactly an M/M/10/10 loss system whose rejection probability is Erlang-B. In the shared design all messages queue for the common pool, and contention stretches each service time by a factor that grows with the number of busy threads:

$$\text{Dedicated: } P_reject = B(N/2, \lambda/\mu), \quad X = \lambda \cdot (1 - B) \tag{4}$$

$$\text{Shared: } S = S_base \cdot (1 + \alpha \cdot N_busy/N), \quad \alpha = 0.1 \tag{5}$$

Table 10: Threading architectures (mean ± 95% CI, 10 replications). Analytical values: M/M/20 for the baseline, Erlang-B for the dedicated pool.

ρ	λ	Baseline resp (s)	M/M/20 exact (s)	Shared resp (s)	Dedicated resp (s)	Dedicated X (msg/s)	Erlang-B X	Rejected
0.5	120	0.0834 ± 0.0001	0.0834	0.0883 ± 0.0001	0.0833 ± 0.0001	94.2	94.3	21.4%
0.6	144	0.0836 ± 0.0001	0.0836	0.0898 ± 0.0002	0.0834 ± 0.0002	100.5	100.5	30.2%
0.7	168	0.0847 ± 0.0001	0.0846	0.0935 ± 0.0001	0.0834 ± 0.0001	104.6	104.6	37.7%
0.8	192	0.0888 ± 0.0002	0.0887	0.1092 ± 0.0007	0.0834 ± 0.0001	107.4	107.4	44.1%
0.9	216	0.1065 ± 0.0009	0.1063	0.5075 ± 0.0838	0.0834 ± 0.0001	109.3	109.4	49.4%

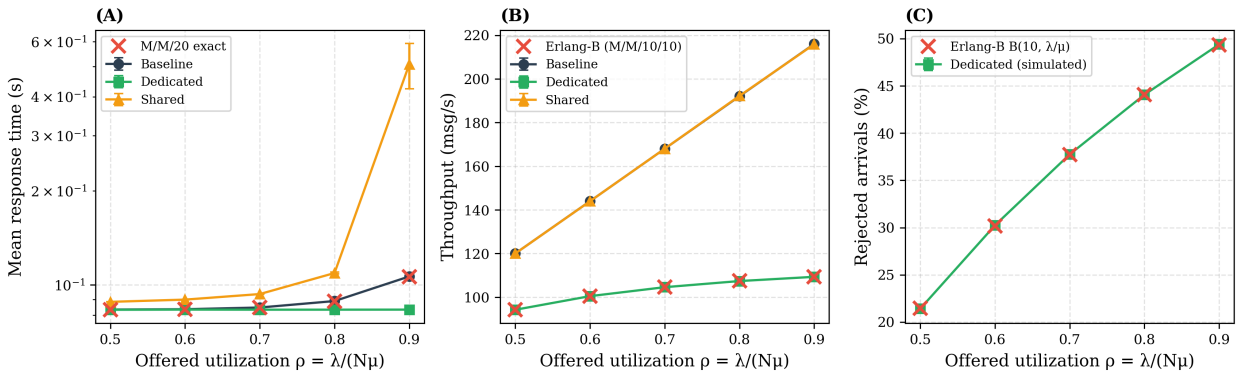


Figure 9: (A) Mean response time, (B) throughput, and (C) rejection rate versus offered utilization; × marks are exact M/M/20 and Erlang-B values.

The simulation reproduces both analytical models: baseline response times match M/M/20 to the fourth decimal, and dedicated throughput and rejection match Erlang-B. The dedicated pool's response time is flat at 1/μ because admitted messages never queue, but it rejects 21.4% of arrivals at ρ = 0.5 and 49.4% at ρ = 0.9, capping throughput at 109.3 msg/s. The shared pool accepts all load but is 10.4% slower than the baseline at ρ = 0.7 and 4.8× slower at ρ = 0.9 (0.508 s versus 0.106 s, P95

1.43 s versus 0.29 s): the contention overhead raises the effective utilization, and near saturation even a 10% service-time inflation causes a disproportionate queueing delay. A dedicated pool therefore suits workloads that must bound latency and can shed load; a shared pool suits workloads that cannot reject requests, provided it is kept well below saturation.

6 Queueing Extensions

6.1 Heterogeneous Servers and Dispatch

Real deployments mix instance types. We model groups of n_i servers with rate μ_i, each group with its own FCFS queue, and a dispatcher that assigns each arrival to a group. Under a static probabilistic split, group i receives a Poisson stream of rate λ_i and is an exact $M/M/n_i$ queue, so the overall mean wait and the stability condition are

$$W_q = \Sigma_i \, (\lambda_i/\lambda) \cdot W_q^{M/M/n_i}(\lambda_i, \mu_i), \quad \text{stable} \Leftrightarrow \lambda_i < n_i \cdot \mu_i \; \forall \, i \qquad (6)$$

Aggregate stability, $\lambda < \Sigma \, n_i \cdot \mu_i$, is not sufficient. We compare per-server random dispatch (each server receives λ/N regardless of speed), capacity-weighted random dispatch ($\lambda_i \propto n_i \cdot \mu_i$, so every group runs at the aggregate ρ), and four dynamic policies, at $\lambda = 50$ msg/s for five configurations of five servers (2000 s runs, 200 s warm-up, 10 replications).

Table 11: Heterogeneous servers (mean wait, s; mean ± 95% CI). ρ_{max} is the largest per-group utilization under per-server random dispatch.

Configuration	Capacity	ρ_{max} (random)	Random (per server)	Random (capacity)	Exact, capacity split	Shortest queue
Homogeneous (5@12)	60	0.83	0.061 ± 0.002	0.061 ± 0.002	0.062	0.060 ± 0.002
Low variance (3@11 + 2@13.5)	60	0.91	0.207 ± 0.009	0.146 ± 0.004	0.146	0.072 ± 0.003
Medium variance (3@10 + 2@15)	60	1.00	4.278 ± 1.804 (unstable)	0.144 ± 0.002	0.146	0.077 ± 0.002
High variance (2@8 + 3@15)	61	1.25	76.401 ± 1.330 (unstable)	0.128 ± 0.005	0.129	0.073 ± 0.002
Very high variance (1@6 + 4@16)	70	1.67	57.675 ± 0.464 (unstable)	0.058 ± 0.003	0.058	0.040 ± 0.001

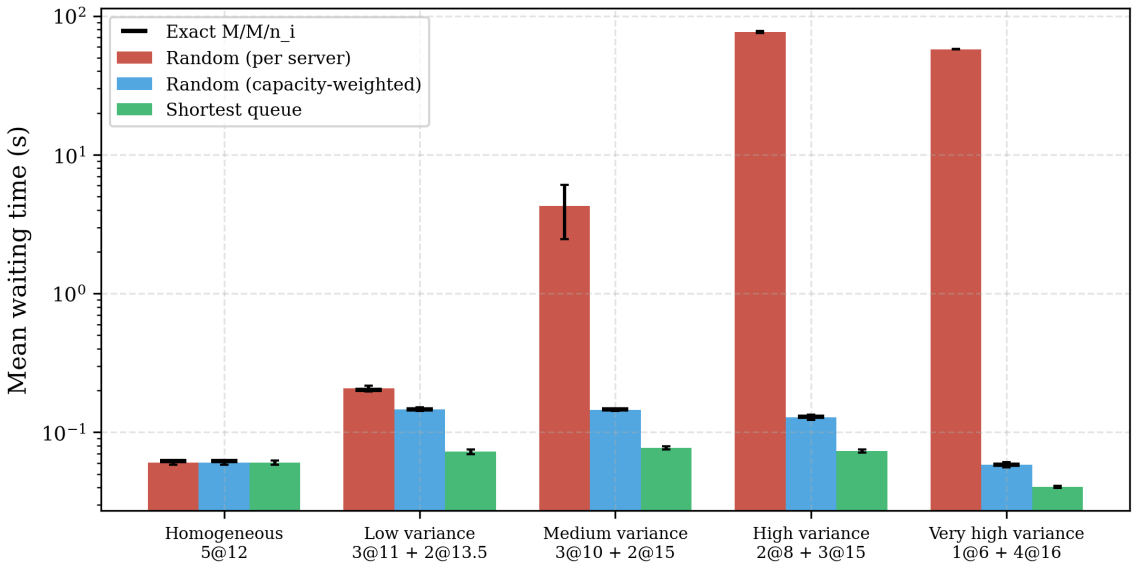


Figure 10: Mean waiting time by configuration and dispatch policy (log scale); black marks are the exact value of Eq. (6) for static policies.

Per-server random dispatch overloads the slow servers whenever λ/N exceeds their rate. That happens for the medium (ρ_max = 1.00), high (1.25), and very high (1.67) variance configurations, even though aggregate utilization is at most 0.83. These queues are unstable, not slow: their measured mean wait grows in proportion to the simulated time (Figure 11; high variance: 19.5, 38.4, 76.4, and 152.4 s for 500–4000 s runs), so any single number reported for them depends only on run length. Where Eq. (6) is finite, simulation agrees with it to within the confidence interval. With capacity-weighted dispatch every configuration is stable and the service-rate spread has little effect: the low- and medium-variance configurations, which have the same group sizes and capacity, have identical exact waits, and the most heterogeneous configuration—which also has the most capacity—has the shortest wait (0.058 s, versus 0.061 s for homogeneous). The residual penalty of the other configurations relative to homogeneous comes from splitting one five-server queue into separate two- and three-server queues, which loses pooling efficiency.

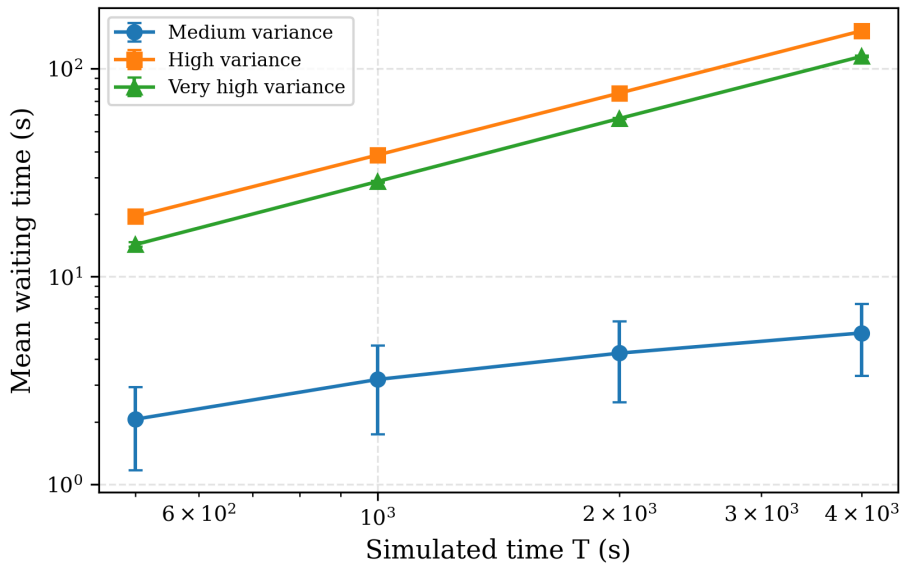


Figure 11: Mean waiting time versus simulated time under per-server random dispatch. Linear growth indicates an unstable group.

Table 12: Dispatch policies on the 2@8 + 3@15 configuration (10 replications).

Policy	Mean wait (s)	P99 wait (s)	P99 response (s)
Random (per server) (unstable)	76.401 ± 1.330	388.061	388.191
Random (capacity-weighted)	0.128 ± 0.005	1.005	1.162
Round-robin (unstable)	77.433 ± 2.011	394.323	394.444
Shortest queue	0.073 ± 0.002	0.503	0.706
Fastest idle first	0.071 ± 0.003	0.559	0.667
Work stealing	0.061 ± 0.002	0.536	0.705

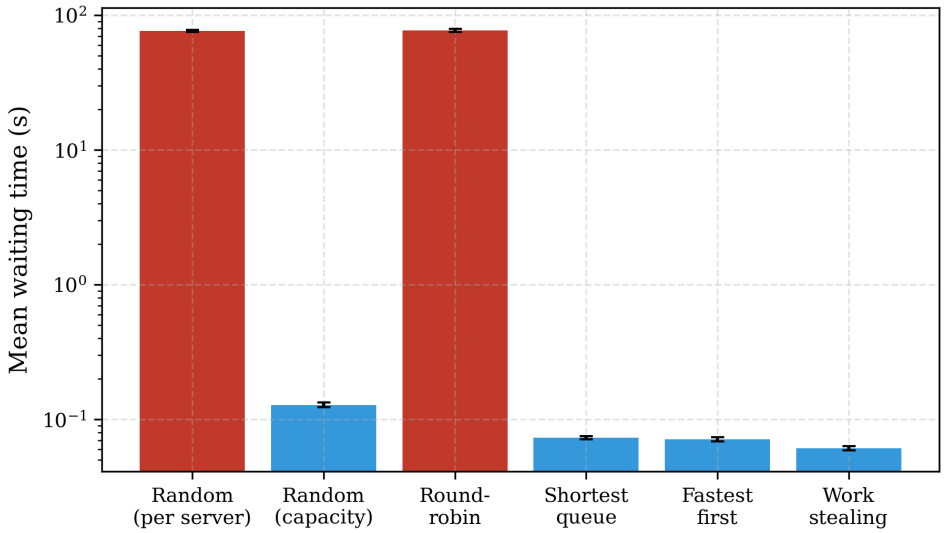


Figure 12: Dispatch policy on the 2@8 + 3@15 configuration (log scale); red bars are unstable.

Among policies (Table 12), round-robin is as unstable as per-server random dispatch because it sends the same λ/N to every server. Capacity-weighted random dispatch is stable but leaves idle fast servers next to busy slow ones. The dynamic policies do best: work stealing, where idle servers take jobs from the longest queue, gives 0.061 s, fastest-idle-first 0.071 s, and shortest queue 0.073 s—close to the 0.061 s of the homogeneous single queue. For mixed fleets the dispatcher, not the hardware mix, decides performance.

6.2 Finite Capacity (M/M/N/K)

A broker with bounded storage rejects arrivals when K messages are in the system. The stationary distribution of the M/M/N/K birth–death chain gives the blocking probability P_K, the admitted rate λ(1 − P_K), and, by Little's Law over admitted messages, the mean wait; for K = N, P_K is the Erlang-B formula. Unlike the infinite-buffer queue, the system is stable for any ρ. We swept K from 10 to 200 with N = 10, μ = 12, and ρ = 0.8, 1.0, and 1.25 (2000 s runs, 200 s warm-up, 20 replications).

Table 13: Finite-capacity queue, simulation (mean ± 95% CI, 20 replications) versus the exact birth–death solution.

ρ	K	Sim P_K	Exact P_K	Sim Wq (s)	Exact Wq (s)
0.8	10	0.1221 ± 0.0007	0.1217	0	0.0000
0.8	20	0.0091 ± 0.0003	0.0091	0.0122 ± 0.0002	0.0121
0.8	50	1.1e-05 ± 1.1e-05	1.1e-05	0.0173 ± 0.0003	0.0170
0.8	200	0.0e+00 ± 0.0e+00	3.2e-20	0.0173 ± 0.0003	0.0170
1	10	0.2149 ± 0.0008	0.2146	0	0.0000

1	20	0.0690 ± 0.0006	0.0682	0.0337 ± 0.0002	0.0336
1	50	0.0225 ± 0.0006	0.0224	0.1564 ± 0.0017	0.1565
1	200	0.0049 ± 0.0009	0.0051	0.7692 ± 0.0416	0.7808
1.25	10	0.3218 ± 0.0007	0.3220	0	0.0000
1.25	20	0.2081 ± 0.0009	0.2085	0.0565 ± 0.0001	0.0564
1.25	50	0.1996 ± 0.0011	0.2000	0.2999 ± 0.0006	0.3000
1.25	200	0.1998 ± 0.0010	0.2000	1.5503 ± 0.0019	1.5500

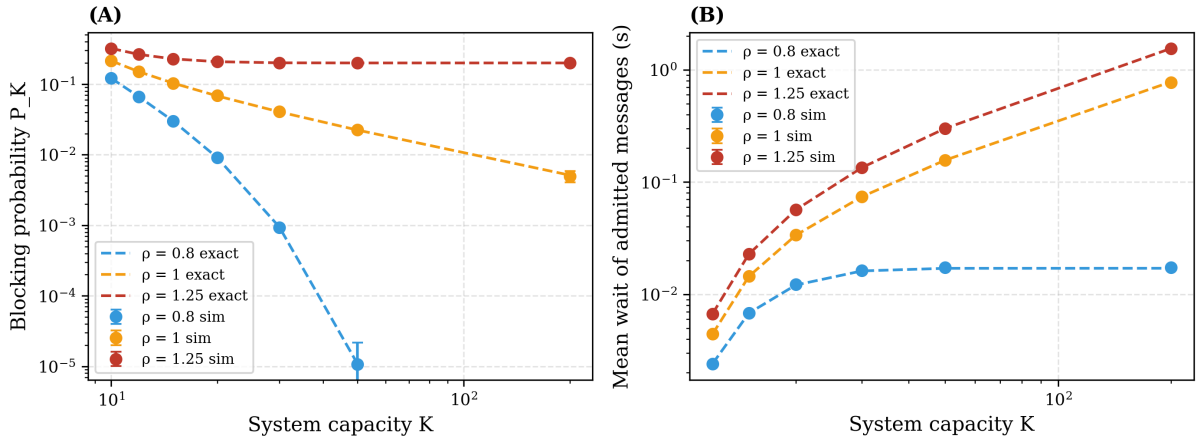


Figure 13: (A) Blocking probability and (B) mean wait of admitted messages versus capacity K; dashed lines are exact.

Simulation agrees with the exact solution: 39 of 42 blocking and waiting-time comparisons fall inside the 95% confidence interval; two of the others differ from the exact value by less than 2%, and the third is a blocking probability of order 10^{-20} that no finite run can observe [21]. The sweep shows the buffer-sizing trade-off. At ρ = 0.8 a modest buffer nearly eliminates loss (P_K = 1.1e-05 at K = 50). In overload (ρ = 1.25) no buffer can help: blocking falls only from 0.322 at K = 10 to the limiting 1 − 1/ρ = 0.200, while the wait of admitted messages grows to 1.55 s at K = 200. In overload a large buffer only converts rejections into delay.

6.3 Priority Queueing

Brokers often serve a high-priority class ahead of standard traffic. With equal exponential service rates, Cobham's formula [23] gives the non-preemptive mean wait of class k as

$$W_k = [C(N, a)/(N\cdot\mu)] / [(1 - \sigma_{k-1})(1 - \sigma_k)] \qquad (7)$$

where C(N, a) is the Erlang-C probability and σ_k is the utilization of classes 1…k. We simulated non-preemptive and preemptive-resume priority with N = 10, μ = 12, a 20%/80% high/low split, and λ = 100 msg/s (ρ = 0.83), plus sweeps over load and high-priority share (2000 s runs, 200 s warm-up, 20 replications).

Table 14: Priority classes at λ = 100 msg/s, 20% high priority (mean ± 95% CI, 20 replications).

Discipline	Class	Sim mean wait (s)	Analytical (s)	Sim P99 wait (s)
Non-preemptive	High	0.00487 ± 0.00005	0.00488	0.0390
Non-preemptive	Low	0.02915 ± 0.00082	0.02926	0.2384
Non-preemptive	All (load-weighted)	0.02429 ± 0.00066	0.02438	0.2247
Preemptive-resume	High	0.00000 ± 0.00000	0.00000	0.0000
Preemptive-resume	Low	0.03022 ± 0.00076	0.03048	0.2618
Preemptive-resume	All (load-weighted)	0.02417 ± 0.00061	0.02438	0.2425

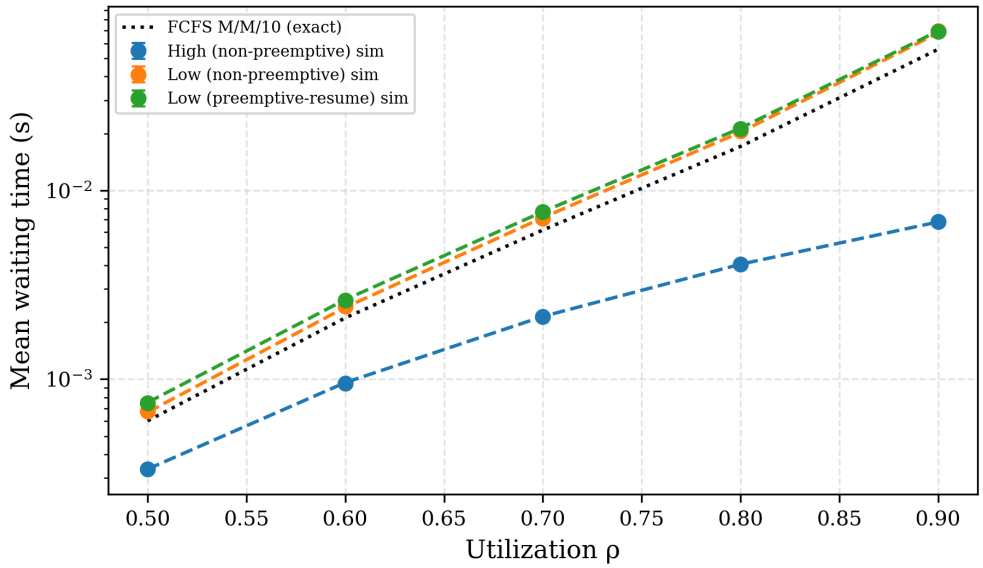


Figure 14: Per-class mean waiting time versus utilization (20% high priority); dashed lines are analytical, dotted is FCFS M/M/10.

Non-preemptive priority cuts the high-priority mean wait to 0.00487 s (Cobham 0.00488 s) and raises the low class to 0.02915 s (Cobham 0.02926 s), against 0.02438 s for FCFS. Preemptive-resume reduces the high-priority wait to effectively zero. In both disciplines the load-weighted mean wait equals the FCFS value to within the confidence interval at every high-priority share, confirming the conservation law: priority redistributes delay between classes without removing any. Simulated means fall inside the 95% confidence interval of the analytical value in all but three cases, all of which are preemptive high-priority rows where the simulation observed no waiting at all against an analytical value below 10^{-6} s.

6.4 Erlang-k Service (M/E_k/N)

Finally we reduce service variability: Erlang-k service is a sum of k exponential phases with rate kμ, so its mean stays 1/μ and C_s² = 1/k. We compare simulation with the Allen–Cunneen approximation [24], Wq ≈ Wq^{M/M/N}·(1 + C_s²)/2, and, for k ≤ 4, with the exact mean wait from a numerically solved CTMC, at ρ = 0.5 (λ = 30, N = 6, μ = 10) and ρ = 0.85 (λ = 51) (5000 s runs, 500 s warm-up, 20 replications).

Table 15: Erlang-k service (mean wait in ms; mean ± 95% CI, 20 replications).

ρ	k	C_s²	Sim Wq (ms)	Exact CTMC (ms)	Allen–Cunneen (ms)	Sim P99 wait (ms)
0.5	1	1	3.315 ± 0.061	3.305	3.305	76.8
0.5	2	0.5	2.697 ± 0.042	2.654	2.479	61.2
0.5	4	0.25	2.328 ± 0.036	2.332	2.065	52.1
0.5	8	0.125	2.157 ± 0.017	—	1.859	48.1
0.5	16	0.0625	2.086 ± 0.028	—	1.756	46.6
0.85	1	1	69.578 ± 1.239	69.339	69.339	463.8
0.85	2	0.5	52.642 ± 0.906	52.793	52.004	341.3
0.85	4	0.25	44.831 ± 0.578	44.490	43.337	286.9
0.85	8	0.125	39.887 ± 0.596	—	39.003	250.6

0.85	16	0.0625	38.177 ± 0.372	—	36.836	238.5

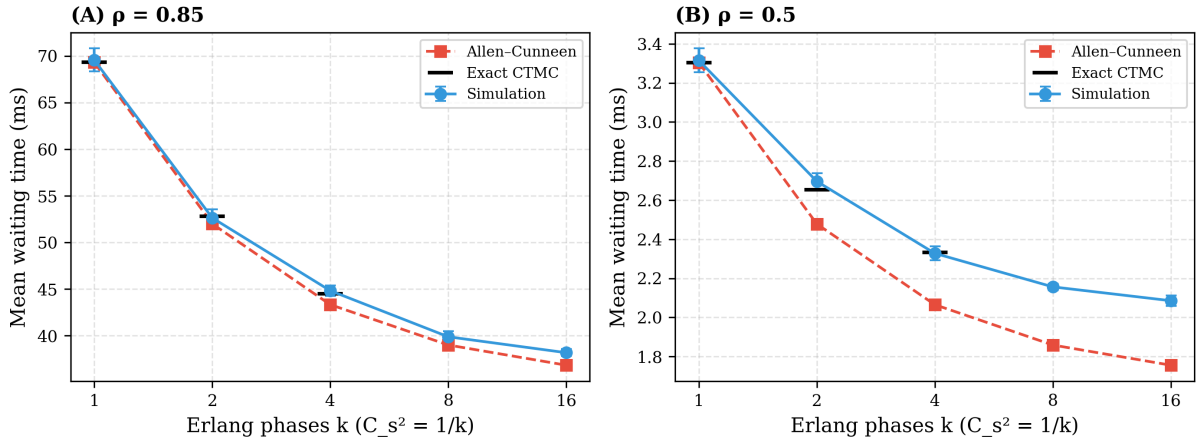


Figure 15: Mean waiting time versus number of Erlang phases at (A) ρ = 0.85 and (B) ρ = 0.5; black marks are exact CTMC values.

Waiting time falls monotonically with k. From k = 1 to k = 16 the mean wait drops by 37% at ρ = 0.5 and 45% at ρ = 0.85, and the P99 wait by 39% and 49%. Simulation matches the exact CTMC to within 1.6% for every k where it was computed. The Allen–Cunneen approximation is exact for k = 1 but underestimates the wait for k ≥ 2—by 11% at ρ = 0.5 and k = 4—and becomes accurate only in heavy traffic. This is the mirror image of the heavy-tail result in our companion paper: there, high service variance inflated tail latency; here, low variance compresses it.

7 Discussion

Table 16: Summary of measured effects (all values from Tables 2–15).

Mechanism	Measured effect	Conditions
Visibility timeout	Optimum near 1 s; 0.5 s → 54% duplicates; 30 s → 3.8 s mean latency	λ = 10, processing 0.5 s, p = 0.1
Replication R = 1 → 2	1st-delivery 4.92 → 0.50 s under outages; no effect without	3 nodes, MTBF 100 s, MTTR 20 s
In-order delivery	+19% mean latency	λ = 80, 10–100 ms reordering
2PC	61 ms/msg; capacity −42%	n = 3, a = 0.99, T = 1 s
Raft commit	Capacity −7% (LAN), −36% (WAN)	n = 3, synchronous commit
Causal delivery	17% held back, ~14 ms each	P = 10, delay CV = 1
Shared thread pool	+10% (ρ = 0.7), 4.8× (ρ = 0.9)	N = 20, α = 0.1
Dedicated thread pool	Rejects 21%–49% (Erlang-B)	N = 20, 2 threads/connection
Heterogeneous servers	Per-server random dispatch unstable; capacity-aware dispatch removes penalty	λ = 50, 5 servers
Erlang-k k = 1 → 16	Mean wait −45%	ρ = 0.85

Three observations cut across the experiments. First, timeouts dominate several costs: the 2PC vote timeout contributes half of 2PC's mean overhead and nearly all of its variance and tail, and the visibility timeout sets both the retry latency and the duplicate rate. Timeout values deserve the same tuning attention as thread counts. Second, the cost of consistency is set by network round trips relative to service time: Raft costs single digits of capacity on a LAN and over a third on a WAN, and in both

2PC and Raft a pool that is comfortably stable without the protocol can become unstable with it, so capacity planning must use the protocol-inflated service time. Third, in several cases the policy matters more than the resource: the dispatcher, not the server mix, determines whether a heterogeneous fleet is stable; a second replica captures nearly all the availability benefit of replication; and the dedicated/shared choice is a choice between rejection and delay.

Limitations. All workloads are synthetic Poisson arrivals with exponential service unless stated, so absolute numbers depend on the parameters in Table 1. The broker model polls storage nodes with a fixed back-off, injects node outages but not network partitions or message loss, and applies acknowledgements to unavailable nodes. The Raft model commits each request synchronously with no batching, pipelining, or disk writes, and is exercised without leader churn during the queueing experiments. The 2PC model does not model coordinator failure. The ordering results at $\rho \approx 0.9$ are inconclusive because of the infinite-variance service distribution. Validation against production broker traces remains future work.

8 Conclusion and Future Work

This paper complements our study of heavy-tailed service times by measuring the cost of the reliability, consistency, and threading mechanisms that production brokers layer on top of the raw queue. Every result is a replicated, seeded simulation measurement, and every model with a known closed form reproduces it. Atomic commit is the most expensive guarantee we measured (42% of capacity with default timeouts), consensus costs whatever the network round trip costs, and causal ordering and second replicas are comparatively cheap. Visibility and vote timeouts are first-order performance parameters, and for heterogeneous fleets the dispatch policy determines stability.

Future work includes simulating network partitions to quantify the availability side of the CAP trade-off, adding entry batching and pipelining to the Raft model, repeating the ordering experiment near saturation with more replications, combining the heavy-tailed service times of our companion paper with 2PC and consensus overheads to test whether workload variance and protocol cost compound, and validating the framework against production broker traces.

References

[1] J. Li, Y. Cui, and Y. Ma, “Modeling message queueing services with reliability guarantee in cloud computing environment using colored Petri nets,” Mathematical Problems in Engineering, vol. 2015, Art. no. 383846, 2015.

[2] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” Communications of the ACM, vol. 21, no. 7, pp. 558–565, 1978.

[3] C. J. Fidge, “Timestamps in message-passing systems that preserve the partial ordering,” in Proc. 11th Australian Computer Science Conf., 1988, pp. 56–66.

[4] F. Mattern, “Virtual time and global states of distributed systems,” in Parallel and Distributed Algorithms, 1989, pp. 215–226.

[5] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in Proc. USENIX Annual Technical Conf. (ATC), 2014, pp. 305–319.

[6] L. Lamport, “The part-time parliament,” ACM Trans. Computer Systems, vol. 16, no. 2, pp. 133–169, 1998.

[7] J. Gray, “Notes on data base operating systems,” in Operating Systems: An Advanced Course, LNCS 60, Springer, 1978, pp. 393–481.

[8] J. Gray and L. Lamport, “Consensus on transaction commit,” ACM Trans. Database Systems, vol. 31, no. 1, pp. 133–160, 2006.

[9] E. A. Brewer, “Towards robust distributed systems,” keynote, ACM Symp. Principles of Distributed Computing (PODC), 2000.

[10] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” ACM SIGACT News, vol. 33, no. 2, pp. 51–59, 2002.

[11] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” Journal of the ACM, vol. 32, no. 2, pp. 374–382, 1985.

[12] W. Vogels, “Eventually consistent,” Communications of the ACM, vol. 52, no. 1, pp. 40–44, 2009.

[13] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. ACM SOSP, 2007, pp. 205–220.

[14] J. Dean and L. A. Barroso, “The tail at scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013.

[15] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York: Wiley-Interscience, 1975.

[16] A. K. Erlang, “Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges,” Elektroteknikeren, vol. 13, pp. 5–13, 1917.

[17] J. D. C. Little, “A proof for the queuing formula: L = λW,” Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

[18] M. Harchol-Balter, Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge: Cambridge Univ. Press, 2013.

[19] Amazon Web Services, “Amazon SQS Developer Guide: Visibility timeout,” 2023. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/

[20] Team SimPy, “SimPy: Discrete-event simulation for Python,” 2023. [Online]. Available: https://simpy.readthedocs.io/

[21] D. Gross and C. M. Harris, Fundamentals of Queueing Theory, 3rd ed. New York: Wiley, 1998.

[22] K. Birman, A. Schiper, and P. Stephenson, “Lightweight causal and atomic group multicast,” ACM Trans. Computer Systems, vol. 9, no. 3, pp. 272–314, 1991.

[23] A. Cobham, “Priority assignment in waiting line problems,” Operations Research, vol. 2, no. 1, pp. 70–76, 1954.

[24] A. O. Allen, Probability, Statistics, and Queueing Theory with Computer Science Applications, 2nd ed. San Diego: Academic Press, 1990.